# Plan de implementación — IconScript

> Compilador de un lenguaje de dibujo tipo Logo para diseñar íconos (apps, archivos, badges), construido con **Coco/R (C#)**. Trabajo Final de Compiladores — UNSJ 2026 — grupo de 3 integrantes.

---

## 0. Prompt inicial para pegar en Claude Code

```
None
Vas a ayudarnos a implementar el compilador IconScript siguiendo el archivo
PLAN-IconScript.md
que está en la raíz del repositorio. Leelo completo antes de empezar.

Reglas obligatorias:
1. Es un trabajo académico: trabajamos por FASES. Al terminar cada fase, frená,
mostrame
   qué hiciste, cómo probarlo y qué conceptos de compiladores aplica. No avances a
la
   siguiente fase hasta que yo lo apruebe.
2. Al final de cada fase agregá una entrada en docs/bitacora-ia.md (formato en la
sección 9).
3. Comentá el código en español explicando la relación con la teoría (First,
LL(1),
   atributos, patching, máquina de pila). Tenemos que poder defenderlo sin IA.
4. Nunca edites a mano Scanner.cs ni Parser.cs: se regeneran con Coco/R desde el
.ATG.
5. Respetá el alcance: nada de texto, tipografías ni imágenes externas.

Empezá por la Fase 0.
```

---

## 1. Reglas de la cátedra que el código debe respetar

| Regla | Cómo se cumple en el proyecto |
|---|---|
| Usar Coco/R para generar scanner y parser | `IconScript.ATG` → `Scanner.cs` + `Parser.cs` (nunca editados a mano) |

| Regla | Cómo se cumple en el proyecto |
|---|---|
| El trabajo debe ser propio; la IA ayuda, no reemplaza | Implementación por fases con revisión humana entre fases |
| Documentar dónde y cómo se usó la IA (herramienta + prompts) | `docs/bitacora-ia.md` actualizado en cada fase |
| No copiar respuestas de IA sin modificación | El grupo revisa, renombra, ajusta y prueba cada fase |
| Verificar críticamente lo generado | Tests automáticos + programas de prueba en cada fase |
| Demostrar comprensión sin IA | Comentarios didácticos + `docs/diseno.md` explicando cada decisión |
| No compartir información sensible | El repo no contiene datos personales ni credenciales |
| Entregables: código, documento de diseño, interfaces, manuales, anexos | Secciones 7 y 8 |

---

## 2. Entorno

- **SO de desarrollo:** Windows 10/11 (también debe compilar en Linux/macOS salvo el paso de Coco.exe, ver abajo).
- **.NET SDK 8** (`dotnet --version` ≥ 8).
- **Coco/R para C#** (versión del 19/04/2011, ya probada con Taste).
  - En la PC de José ya existe en `C:\compi\taste\Coco.exe` + `Scanner.frame` + `Parser.frame`.
  - Copiarlos a `tools/coco/` del repo.
  - **Opción preferida (multiplataforma):** descargar `CocoSourcesCS.zip` de https://ssw.jku.at/Research/Projects/Coco/ y crear `tools/CocoR/CocoR.csproj` (console, net8.0) con esas fuentes, para correr Coco con `dotnet run --project tools/CocoR -- ...`. Si da problemas, usar `Coco.exe` en Windows.
- **Node 20+** para la web (Vite + React + TypeScript).
- Lecciones aprendidas con Taste:
  - Coco necesita `-namespace IconScript` para que el código generado compile con el resto.

- Los warnings de nulabilidad (CS8618/CS8625) del código generado se silencian con `<Nullable>disable</Nullable>` en el `.csproj` del Core.
- Coco/R informa conflictos LL(1) como *warnings*: hay que analizarlos y documentarlos, no ignorarlos.

---

## 3. Estructura del repositorio

```
None
IconScript/
├── PLAN-IconScript.md
├── README.md                      ← manual de uso (instalación, comandos,
ejemplos)
├── docs/
│   ├── bitacora-ia.md             ← bitácora obligatoria de uso de IA
│   ├── diseno.md                  ← documento de diseño (gramática, IR, VM,
decisiones)
│   └── gramatica-ebnf.md          ← gramática limpia, sin acciones
├── tools/
│   ├── coco/                      ← Coco.exe, Scanner.frame, Parser.frame (frames
MODIFICADOS, ver 5.6)
│   └── CocoR/                     ← (opcional) Coco compilado con dotnet
├── src/
│   ├── IconScript.Core/           ← biblioteca: TODO el compilador
│   │   ├── IconScript.ATG
│   │   ├── Scanner.cs  Parser.cs ← GENERADOS por Coco (commiteados igual)
│   │   ├── SymbolTable.cs         ← tabla de símbolos con tipos
│   │   ├── Types.cs               ← enum IconType { Undef, Number, Color }
│   │   ├── Palettes.cs            ← paletas predefinidas
│   │   ├── CodeGen.cs             ← Op, Instr, emisión, patching, listado
│   │   ├── TurtleVM.cs            ← máquina virtual de la tortuga
│   │   ├── Geometry.cs            ← cálculo de ARC, bounding box, espejo
│   │   ├── Templates.cs           ← app / file / badge
│   │   ├── SvgWriter.cs           ← trazos → SVG
│   │   └── Compiler.cs            ← fachada: Compile(string fuente) →
CompileResult
│   ├── IconScript.Cli/            ← ejecutable `iconc`
│   └── IconScript.Api/            ← ASP.NET Minimal API (POST /compile)
├── web/                           ← editor React (Vite + TS)
├── examples/                      ← programas .icon de ejemplo y de error
└── tests/IconScript.Tests/        ← xUnit
```

Regeneración del parser: target MSBuild `BeforeBuild` en `IconScript.Core.csproj` que ejecute Coco **solo si el .ATG cambió** (Inputs/Outputs), o un script `scripts/gen.ps1` + `scripts/gen.sh`:

```
None
Coco IconScript.ATG -namespace IconScript -frames ../../tools/coco
```

---

# 4. Especificación del lenguaje

## 4.1 Léxico

| Token | Definición (Cocol/R) | Ejemplo |
|---|---|---|
| number | digit {digit} ["." digit {digit}] | 100, 2.5 |
| color | '#' hex hex hex hex hex hex | #4F46E5 |
| ident | letter {letter \| digit \| '_'} | ocean, app |
| varDecl | '"' letter {letter \| digit \| '_'} | "lado (para MAKE) |
| varRef | ':' letter {letter \| digit \| '_'} | :lado |
| Comentarios | COMMENTS FROM "//" TO lf y COMMENTS FROM "/*" TO "*/" NESTED | |
| Ignorar | IGNORE cr + lf + tab | |
| Mayúsculas | IGNORECASE (FD = fd = Fd) | |

Palabras clave (literales en las producciones): TEMPLATE FD FORWARD BK BACK RT RIGHT LT LEFT PU PENUP PD PENDOWN ARC MOVETO REPEAT MIRROR COLOR WIDTH PALETTE SHADOW BACKGROUND GRADIENT FILL CIRCLE RECT MAKE NOAUTOFIT.

## 4.2 Sintaxis (EBNF de referencia — sin atributos)

```
None
IconScript  = [ TemplateDecl ] { Statement } .
TemplateDecl= "TEMPLATE" ident .                      // app | file | badge
(chequeo semántico)
Statement   = Move | Turn | Pen | Arc | MoveTo | Repeat | Mirror
            | ColorCmd | WidthCmd | PaletteCmd | ShadowCmd
            | Background | Fill | Circle | Rect | Make | NoAutofit .
Move        = ( "FD" | "FORWARD" | "BK" | "BACK" ) Expr .
Turn        = ( "RT" | "RIGHT" | "LT" | "LEFT" ) Expr .
Pen         = "PU" | "PENUP" | "PD" | "PENDOWN" .
Arc         = "ARC" Expr Expr .                         // radio, grados
MoveTo      = "MOVETO" Expr Expr .                      // x, y (relativos al
centro)
Repeat      = "REPEAT" Expr "[" { Statement } "]" .
Mirror      = "MIRROR" "[" { Statement } "]" .
ColorCmd    = "COLOR" Expr .
WidthCmd    = "WIDTH" Expr .
PaletteCmd  = "PALETTE" ident .
ShadowCmd   = "SHADOW" .
Background  = "BACKGROUND" Paint .
Fill        = "FILL" Paint "[" { Statement } "]" .
Paint       = "GRADIENT" Expr Expr | Expr .
Circle      = "CIRCLE" Expr .
Rect        = "RECT" Expr Expr .
Make        = "MAKE" varDecl Expr .
NoAutofit   = "NOAUTOFIT" .
Expr        = Term { ( "+" | "-" ) Term } .
Term        = Factor { ( "*" | "/" ) Factor } .
Factor      = number | color | varRef | "(" Expr ")" | "-" Factor .
```

**Conflicto LL(1) conocido (documentarlo, es material para la defensa):** en comandos con dos expresiones (`ARC 30 -180`), el `-` puede continuar la primera expresión (resta) o empezar la segunda (negativo). Coco/R lo informa como warning y elige la resta. **Decisión:** se acepta el comportamiento por defecto (igual que el *else colgante* de Taste) y se documenta que un segundo argumento negativo se escribe entre paréntesis: `ARC 30 (-180)`. Agregar un test que lo verifique y una explicación en `docs/diseno.md`. Cualquier otro warning LL(1) debe eliminarse o justificarse.

## 4.3 Semántica

**Tipos:** `Number` y `Color`. Cada `Expr` devuelve su tipo como atributo de salida: `Expr<out IconType t>`.

| Regla semántica | Mensaje de error |
|---|---|
| `+ - * /` y `-` unario solo entre `Number` | `operadores aritméticos requieren números` |
| `FD BK RT LT WIDTH ARC MOVETO REPEAT CIRCLE RECT` requieren `Number` | `se esperaba un número` |
| `COLOR`, `BACKGROUND`, `FILL`, `GRADIENT` requieren `Color` | `se esperaba un color` |
| `:x` debe estar declarada (por `MAKE` o `PALETTE`) antes de usarse | `variable 'x' no declarada` |
| Una variable no cambia de tipo al reasignarse | `'x' es de tipo color, no se le puede asignar un número` |
| `TEMPLATE` solo `app`, `file` o `badge`, una vez y al inicio | `plantilla desconocida 'x'` |
| `PALETTE` solo nombres existentes | `paleta desconocida 'x'` |
| Constantes: radio de `ARC`/`CIRCLE` > 0, `WIDTH` > 0, `REPEAT` ≥ 0 | `el radio debe ser mayor que 0`, etc. (cuando el valor es constante; si depende de variables, se verifica en ejecución) |
| División por cero con constantes | `división por cero` |

**Tabla de símbolos** (`SymbolTable.cs`), modelo de la materia:

- `Symbol { string Name; IconType Type; int Slot; }` — `Slot` = índice en el arreglo de variables de la VM.
- Operaciones: `Insert(name, type)`, `Find(name)` (devuelve `undefSym` y reporta error si no existe, para evitar cascadas), `Lookup` sin error.
- Un solo scope global (no hay procedimientos en el alcance base). Dejar `OpenScope/CloseScope` implementados pero documentados como extensión futura.
- `PALETTE ocean` inserta (o reasigna) `primary`, `secondary`, `accent`, `light`, `dark` de tipo `Color`.

**Paletas** (`Palettes.cs`) — 5 paletas: `ocean`, `sunset`, `forest`, `candy`, `mono`, cada una con los 5 colores anteriores. Elegir colores armónicos (por ejemplo, basados en escalas tipo Tailwind).

### 4.4 Ejemplos que deben compilar

```
None
// examples/flor.icon
TEMPLATE app
PALETTE ocean
BACKGROUND GRADIENT :primary :secondary
COLOR :light
REPEAT 8 [ FILL :light [ ARC 120 45 RT 135 ARC 120 45 ] RT 45 ]
```

```
None
// examples/estrella.icon
TEMPLATE badge
BACKGROUND #0F172A
COLOR #FACC15
WIDTH 24
REPEAT 5 [ FD 300 RT 144 ]
```

```
None
// examples/archivo-iconscript.icon  (ícono de archivo del propio lenguaje)
TEMPLATE file
MAKE "c #6366F1
BACKGROUND #FFFFFF
FILL :c [ CIRCLE 140 ]
COLOR #FFFFFF
WIDTH 20
MIRROR [ ARC 80 90 ]
```

Más ejemplos: cuadrado, círculo, corazón, ola, espiral con variables, y **programas con errores** (léxico, sintáctico, semántico) en `examples/errores/`.

---

## 5. Diseño del back-end

### 5.1 Código intermedio (`CodeGen.cs`)

- Representación: `List<Instr>` donde `Instr { Op Op; double Arg; }`; la dirección de una instrucción es su índice.
- Valores en la pila: `double`. Los colores se codifican como `0xRRGGBB` en un double (el tipo ya fue verificado en compilación).

- `Emit(op[, arg])` devuelve la dirección; `Patch(addr, target)` completa saltos hacia adelante.
- `Listing()` devuelve el texto `0: PUSH 8` … para `iconc -code` y la pestaña de la web.

| Op | Pila (antes → después) | Efecto |
|---|---|---|
| `PUSH v` | … → …, v | Constante |
| `LOAD s` / `STORE s` | … ↔ …, var[s] | Variables |
| `ADD SUB MUL DIV NEG` | … | Aritmética (DIV por 0 en ejecución → error) |
| `MOVE` | …, d → … | Avanza d (BK = `NEG` + `MOVE`) |
| `TURN` | …, g → … | Gira g a la derecha (LT = `NEG` + `TURN`) |
| `PENUP PENDOWN` | — | Lápiz |
| `ARC` | …, r, g → … | Arco (g > 0 derecha) |
| `MOVETO` | …, x, y → … | Posición absoluta (sin trazo) |
| `SETCOLOR SETWIDTH` | …, v → … | Estilo |
| `SHADOW` | — | Activa sombra |
| `BG` | …, c → … | Fondo sólido |
| `BGGRAD` | …, c1, c2 → … | Fondo degradado |
| `FILLBEGIN` | …, c → … | Abre bloque de relleno |
| `FILLGRADBEGIN` | …, c1, c2 → … | Abre relleno degradado |
| `FILLEND` | — | Cierra el relleno |
| `CIRCLE` | …, r → … | Círculo centrado en la tortuga |
| `RECT` | …, w, h → … | Rectángulo centrado, rotado según la dirección |
| `LOOPBEGIN a` | …, n → … | Si n ≤ 0 salta a `a`; si no, apila contador en la pila de bucles |

| Op | Pila (antes → después) | Efecto |
|---|---|---|
| `LOOPEND a` | — | Decrementa contador; si > 0 salta a `a` (inicio del cuerpo); si no, lo desapila |
| `MIRRORBEGIN` | — | Guarda el estado de la tortuga |
| `MIRROREND a` | — | 1ª pasada: restaura el estado, activa modo espejo y salta a `a`; 2ª pasada: desactiva el espejo |
| `HALT` | — | Fin |

**Patching:** `REPEAT` emite `LOOPBEGIN 0`, guarda la dirección, genera el cuerpo, emite `LOOPEND inicioCuerpo` y hace `Patch` del `LOOPBEGIN` con la dirección siguiente. Mismo esquema que el `while` de Taste (`FJMP` + `Patch`). Comentarlo así en el código.

**Metadatos fuera del IR:** plantilla elegida, `NOAUTOFIT`.

## 5.2 Máquina virtual (`TurtleVM.cs`)

- Estado: `x, y` (origen = centro del lienzo, y hacia arriba), `heading` en grados (0 = norte, RT suma en sentido horario), `penDown` (inicia en true), `color` (#000000), `width` (8), `mirror` (bool), `shadow`.
- Estructuras: pila de valores, pila de contadores de bucle, pila de estados guardados para `MIRROR`, arreglo de variables.
- En modo espejo: `TURN` y el sentido de `ARC` se invierten (reflexión respecto del eje de la dirección inicial del bloque).
- Salida: `List<Shape>` donde `Shape` = `StrokePath` (segmentos y arcos con color y grosor), `FilledPath` (camino cerrado con pintura), `Circle`, `Rect`, más el fondo.
- Trazos consecutivos con el mismo estilo y lápiz bajo se acumulan en el mismo `StrokePath`.
- Dentro de `FILLBEGIN … FILLEND`, todo movimiento (con lápiz bajo o alto) se agrega al camino del relleno; al cerrar se agrega `Z`. `CIRCLE`/`RECT` dentro de un `FILL` se rellenan con esa pintura.
- **Límite de seguridad:** máximo 1.000.000 de instrucciones ejecutadas y 50.000 formas; si se supera → error de ejecución "programa demasiado largo" (necesario para la vista previa en vivo).
- Errores de ejecución con mensaje claro (radio ≤ 0 calculado, división por cero).

## 5.3 Geometría (`Geometry.cs`)

`ARC r g` desde posición P con dirección h:

1. Vector derecho = (cos(h−90°), sin(h−90°)) en coordenadas de la tortuga; centro C = P + r·derecho si g > 0, P − r·derecho si g < 0.
2. Punto final = rotar P alrededor de C en −g (horario si g > 0).
3. Nueva dirección = h + g.
4. SVG: `A r r 0 largeArc sweep x y` con `largeArc = |g| > 180`, `sweep = g > 0` (horario en pantalla).
5. |g| ≥ 360 → partir en arcos de ≤ 180° (un solo `A` no puede dibujar un círculo completo).
6. Bounding box del arco: extremos + los puntos cardinales del círculo que caen dentro del barrido.

Tests de geometría obligatorios: `REPEAT 4 [FD 100 RT 90]` vuelve al origen con la misma dirección; `ARC 50 360` vuelve al origen; `ARC 50 90` deja la tortuga en (50, 50) con dirección 90° partiendo de (0,0) mirando al norte.

## 5.4 Plantillas (`Templates.cs`)

| **Plantilla** | **Lienzo** | **Forma de fondo / máscara** | **Zona segura (autofit)** |
|---|---|---|---|
| `app` (por defecto) | 1024×1024 | Cuadrado con esquinas redondeadas (rx ≈ 22,5 % del lado) | 60 % central (compatible con íconos adaptativos de Android) |
| `file` | 1024×1024 | Hoja con esquina superior derecha doblada | 55 % central, desplazada hacia abajo |
| `badge` | 1024×1024 | Círculo | 65 % central |

**Autofit:** al terminar la ejecución, calcular el bounding box de todas las formas (sin el fondo), trasladar para centrarlo y escalar **solo hacia abajo** si excede la zona segura. `NOAUTOFIT` lo desactiva.

## 5.5 SVG (`SvgWriter.cs`)

- `<svg viewBox="0 0 1024 1024">` con transformación de coordenadas (y hacia arriba → y hacia abajo).

- `<defs>`: `clipPath` con la forma de la plantilla, `linearGradient` (diagonal 135°) por cada degradado usado, `filter` con `feDropShadow` si hay `SHADOW`.
- Fondo = forma de la plantilla rellena. Primer plano dentro de `<g clip-path="...">`.
- Trazos con `stroke-linecap="round" stroke-linejoin="round" fill="none"`.
- Números con un máximo de 2 decimales; SVG válido y determinístico (mismo programa → mismo SVG, para los tests snapshot).

### 5.6 Errores

- Modificar las copias de `Scanner.frame`/`Parser.frame` en `tools/coco/` (documentarlo) para que la clase `Errors` guarde una lista `List<Diagnostic>` con `{ Line, Col, Message, Kind (Lexico|Sintactico|Semantico|Ejecucion) }` en vez de solo imprimir.
- Traducir los mensajes sintácticos generados por Coco ("X expected" → "se esperaba X") en la fachada.
- `Scanner` debe construirse desde un `Stream` (`new Scanner(new MemoryStream(UTF8 bytes))`) para compilar texto de la web sin archivos.
- Si hay errores de compilación, no se ejecuta la VM ni se genera SVG.

### 5.7 Fachada (`Compiler.cs`)

```
C#
public record Diagnostic(int Line, int Col, string Message, string Kind);
public record CompileResult(bool Success, IReadOnlyList<Diagnostic> Errors,
                            string? IntermediateCode, string? Svg, string
Template);
public static class IconCompiler { public static CompileResult Compile(string
source); }
```

---

## 6. Interfaces

### 6.1 CLI `iconc` (`IconScript.Cli`)

```
None
iconc flor.icon                → genera flor.svg
iconc flor.icon -o salida.svg
iconc flor.icon -code          → además imprime el código intermedio
iconc flor.icon -open          → abre el SVG en el navegador
```

- Arrastrar un `.icon` sobre `iconc.exe` (un solo argumento sin flags) = compilar + abrir automáticamente (`Process.Start` con `UseShellExecute = true`), y pausar la consola si hubo errores.
- Errores con formato `línea 3, col 5: se esperaba un número` y código de salida ≠ 0.

### 6.2 API (`IconScript.Api`)

- ASP.NET Minimal API: `POST /compile` body `{ "source": "..." }` → `CompileResult` en JSON.
- `GET /examples` → lista de ejemplos de `examples/`.
- CORS habilitado para `http://localhost:5173`. Tiempo máximo por compilación (ej. 2 s).

### 6.3 Editor web (`web/`, Vite + React + TypeScript)

| **Zona** | **Contenido** |
|---|---|
| Izquierda | Editor CodeMirror 6 con resaltado de sintaxis de IconScript y subrayado de la línea con error |
| Derecha | Vista previa del SVG; selector de máscara (iOS, Android círculo, Android squircle); modo "pantalla de inicio" con el ícono entre íconos falsos |
| Abajo | Panel de errores (clic → va a la línea) |
| Pestañas | Vista previa · Código intermedio · Ayuda (referencia del lenguaje) |
| Barra | Menú de ejemplos · Descargar SVG · Exportar PNG · Exportar paquete de app |

- Compilación en vivo con *debounce* de 300 ms.
- Exportación en el navegador (SVG → `<canvas>` → PNG; ZIP con JSZip):
  - PNG sueltos: 16, 32, 64, 128, 256, 512, 1024.
  - Paquete de app: `icon.png` (1024), `adaptive-icon.png` (solo primer plano, fondo transparente, 1024), `android/mipmap-{mdpi,hdpi,xhdpi,xxhdpi,xxxhdpi}/ic_launcher.png` (48/72/96/144/192), `ios/AppIcon.appiconset/` (1024 + `Contents.json`).

- Verificar los tamaños vigentes de Android/iOS/Expo antes de implementar el paquete (pueden haber cambiado).

---

## 7. Fases de implementación

Prioridad: **P0** = imprescindible para aprobar · **P1** = esperado · **P2** = extra. Si falta tiempo se recorta P2, después P1, nunca P0.

| Fase | Prioridad | Objetivo | Criterio de terminado |
|---|---|---|---|
| **0. Setup** | P0 | Solución .NET (Core, Cli, Api, Tests), Coco integrado al build, `docs/bitacora-ia.md` creado | `dotnet build` regenera el parser desde el .ATG y compila |
| **1. Léxico + sintaxis** | P0 | `IconScript.ATG` sin acciones semánticas (solo tokens y producciones) | Todos los `examples/*.icon` se reconocen; los de `errores/` dan errores sintácticos con línea y columna; warnings LL(1) analizados y documentados |
| **2. Semántica + tabla de símbolos** | P0 | Atributos `<out IconType>`, `SymbolTable`, paletas, todas las reglas de 4.3 | Tests de cada error semántico pasan |
| **3. Código intermedio** | P0 | `CodeGen`, emisión desde las acciones, patching de `REPEAT`, `iconc -code` | Listado correcto para los ejemplos (tests *golden*) |
| **4. VM + SVG mínimo** | P0 | `FD BK RT LT PU PD REPEAT COLOR WIDTH` → SVG | `estrella.icon` produce un SVG |

| Fase | Prioridad | Objetivo | Criterio de terminado |
|---|---|---|---|
| | | | correcto. **Hito de demo** |
| **5. Geometría** | P0 | `ARC`, `MOVETO`, `CIRCLE`, `RECT` | Tests de geometría de 5.3 pasan |
| **6. Diseño de íconos** | P1 | Plantillas, `BACKGROUND`, `GRADIENT`, `PALETTE`, `SHADOW`, `FILL`, `MIRROR`, autofit | `flor.icon` y `archivo-iconscript.icon` se ven bien |
| **7. CLI completa** | P1 | Flags, arrastrar y soltar, mensajes en español | Manual de uso del CLI en README |
| **8. API + editor web** | P1 | 6.2 y 6.3 sin exportación | Edición en vivo con errores y código intermedio |
| **9. Exportación** | P2 | PNG y paquete de app | ZIP descargado con la estructura indicada |
| **10. Cierre** | P0 | Galería de ≥ 10 ejemplos, `docs/diseno.md`, README, tests verdes | Checklist de la sección 8 completo |

En cada fase: escribir primero los tests o programas de prueba, implementar, correr todo y **frenar para revisión humana**.

---

## 8. Checklist de entrega

- ☐ `dotnet test` en verde (léxico, sintaxis, semántica, IR, geometría, snapshots SVG).
- ☐ `examples/` con ≥ 10 íconos y ≥ 6 programas con errores (2 léxicos, 2 sintácticos, 2 semánticos).

- [ ] `docs/gramatica-ebnf.md` y `docs/diseno.md`: tokens, gramática, conflictos LL(1) y su resolución, tabla de símbolos, IR, VM, geometría del ARC, plantillas.
- [ ] README = manual de uso: instalación, CLI, web, referencia del lenguaje con ejemplos.
- [ ] `docs/bitacora-ia.md` completo (sección 9).
- [ ] Ejemplos de íconos originales: **no** recrear logos de marcas o lenguajes existentes.
- [ ] Capturas para el informe y el video.

---

## 9. Bitácora de IA (`docs/bitacora-ia.md`)

Una entrada por fase, con este formato (alimenta el apartado "Reflexiones sobre el uso de la IA generativa" del informe):

```
None
### Fase N — <nombre> — <fecha>
- **Herramienta:** Claude Code (Anthropic)
- **Prompt(s) principales:** "<texto o resumen>"
- **Qué generó la IA:** <archivos / funciones>
- **Qué revisó y cambió el grupo:** <completa el grupo>
- **Errores de la IA detectados:** <código de error, fragmento, cómo se resolvió>
- **Conceptos de la materia aplicados:** <First/Follow, atributos, patching, …>
```

---

## 10. Fuera de alcance

Texto y tipografías, imágenes externas, procedimientos `TO … END` (extensión posible si sobra tiempo: requeriría scopes en la tabla de símbolos y `CALL`/`RET` en la VM), animaciones, edición visual con el mouse.